



Registro de Animales Perdidos

Documento consolidado del proyecto académico

Joisi Torres Vieira

Docente: Ricardo Zamorano

Institución: AIEP, sede San Joaquín

23 de septiembre de 2026

Documento basado en el contenido real del repositorio revisado.

1 1. Problemática, objetivo y alcance

1.1 Problemática

Cuando se pierde una mascota, los datos pueden quedar en papeles o publicaciones separadas. Por eso después cuesta encontrar la información y mantenerla actualizada. Este proyecto busca guardar esos datos en un solo sistema de escritorio conectado a una base de datos.

1.2 Objetivo

Crear un sistema que permita registrar, consultar, actualizar y eliminar datos de animales perdidos, relacionándolos con la persona responsable.

1.3 Alcance real observado

En el repositorio hay una solución C#/NET Windows Forms separada en UI, BLL, DAL y Entidades. La base SQL Server tiene las tablas *Personas* y *AnimalesPerdidos*, unidas por *IdPersona*. La pantalla permite registrar, consultar, seleccionar, actualizar, eliminar y limpiar datos.

No se incluyen usuarios con contraseña, portal web, fotografías, mapas, notificaciones ni reportes, porque esas funciones no aparecen en el código. La revisión se hizo sobre el commit `ae5de9d344444e779b095343509c0efa0cac4753` y no se cambió el repositorio original.

2 2. Requerimientos funcionales

Cada fila está escrita con: **Rol/Usuario + Necesidad/Acción + Criterio de aceptación.**

ID	Rol/Usuario	Necesidad/Acción	Criterio de aceptación
RF01	Mantenedor	Agregar un animal con nombre, especie, características, fecha, lugar, estado e <i>IdPersona</i> .	Se validan los datos obligatorios, se ejecuta <i>InsertarAnimal</i> y se informa el resultado.
RF02	Mantenedor	Leer los animales registrados desde la grilla de la interfaz.	<i>Consultar()</i> obtiene los registros mediante <i>ListarAnimales</i> ; si falla, se muestra un mensaje.
RF03	Mantenedor	Seleccionar una fila para cargar sus datos en el formulario.	El clic copia el identificador, nombre, especie, raza, color, descripción, lugar y estado.
RF04	Mantenedor	Actualizar un animal seleccionado.	Se exige un <i>IdAnimal</i> válido, se valida y se ejecuta <i>ActualizarAnimal</i> ; luego se refresca la grilla.
RF05	Mantenedor	Eliminar un registro existente.	Se exige un identificador válido y confirmación "Sí/No" antes de ejecutar <i>EliminarAnimal</i> .
RF06	Mantenedor	Validar la información ingresada.	Se rechazan datos nulos, nombre, especie, lugar o

ID	Rol/Usuario	Necesidad/Acción	Criterio de aceptación
			estado vacíos, e IdPersona inválido o no numérico.

3 3. Requerimientos no funcionales

ID	Requisito	Descripción
RNF01	Integridad	Mantener la relación Personas-AnimalesPerdidos mediante IdPersona y validar campos obligatorios.
RNF02	Mantenibilidad	Conservar la separación UI, BLL, DAL y Entidades.
RNF03	Manejo de errores	Mostrar mensajes comprensibles sin exponer credenciales, SQL ni detalles internos.
RNF04	Disponibilidad	Permitir las operaciones cuando la aplicación y SQL Server estén disponibles.
RNF05	Usabilidad	Mantener campos identificables, botones CRUD, grilla y confirmación antes de eliminar.
RNF06	Consistencia tecnológica	Mantener compatibilidad con .NET, Windows Forms, SQL Server y Microsoft.Data.SqlClient.

4 4. Requerimientos fortalecidos con ISO/IEC 27001

ISO/IEC 27001:2022 entrega una guía para cuidar la información y sus riesgos de confidencialidad, integridad y disponibilidad [2]. Estas ideas sirven para mejorar el proyecto, pero no quiere decir que esté certificado ni que estos controles estén implementados.

ID	Aspecto	Requerimiento fortalecido y medida verificable
ISO-R01	Acceso	Restringir las operaciones CRUD al mantenedor autorizado y agregar autenticación antes de publicar el sistema.
ISO-R02	Integridad	Conservar validaciones, tipos correctos, procedimientos almacenados y clave foránea.
ISO-R03	Eliminación	Mantener la confirmación antes de borrar y probar la cancelación y la eliminación autorizada.
ISO-R04	Errores	No mostrar cadenas de conexión, credenciales, SQL ni trazas internas al usuario.
ISO-R05	Disponibilidad	Respaldar la base, probar restauración y documentar la respuesta ante caída de SQL Server.
ISO-R06	Cambios	Versionar y revisar cambios en código, procedimientos y esquema antes de integrarlos.
ISO-R07	Privacidad	Tratar teléfono y correo como datos de contacto y limitar su exposición a la gestión necesaria.

5 5. Diagrama de casos de uso del mantenedor

En este diagrama, **Mantenedor** es el usuario encargado de usar el sistema. Se muestran solo las acciones que aparecen en la pantalla y en AnimalBLL.

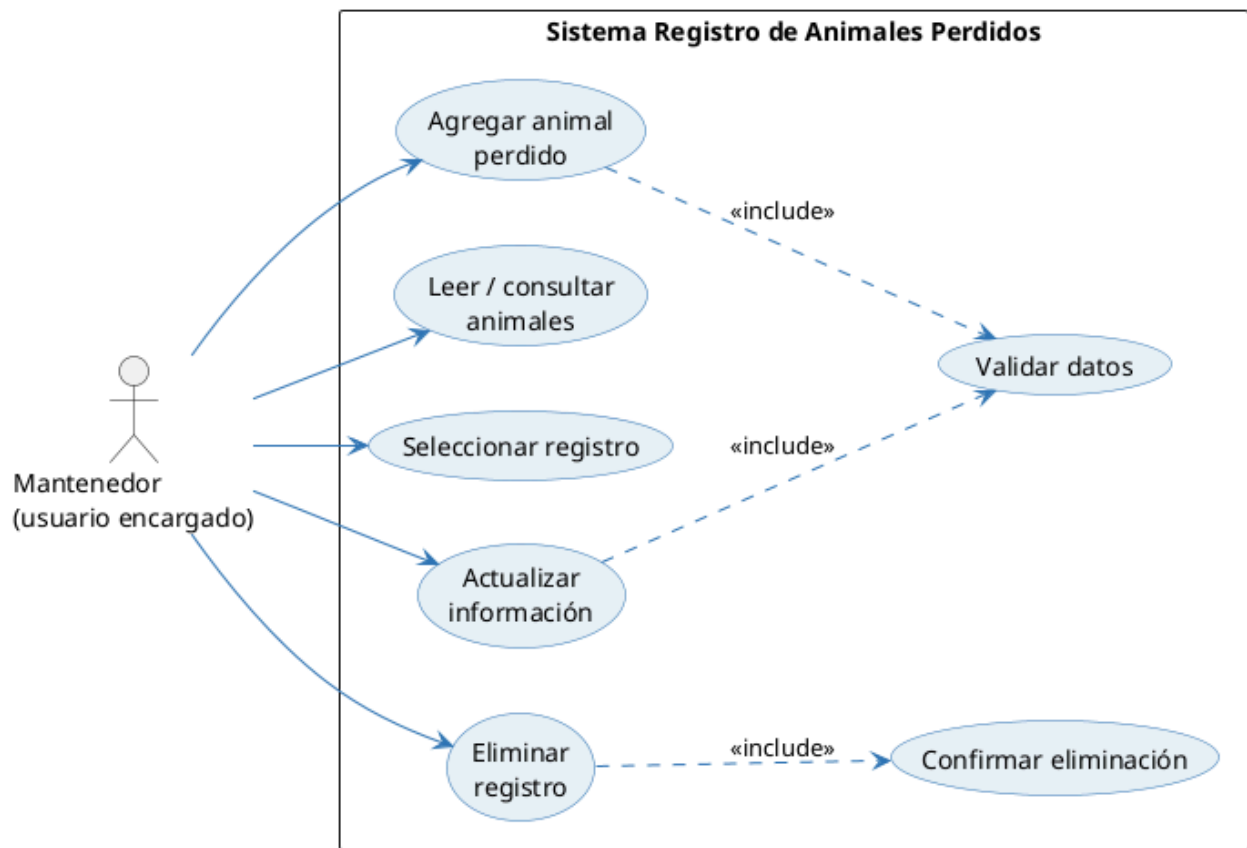


Figura 1. Casos de uso del mantenedor.

6 6. Diagrama de clases completo

El modelo reúne las clases que aparecen en el código: Form1, AnimalBLL, AnimalDAL, Conexion, AnimalPerdido y Persona. Mantenedor se agrega como clase conceptual porque el trabajo la solicita; no existe como archivo propio en el repositorio. Sus operaciones están repartidas entre UI, BLL y DAL.

Diagrama de clases completo

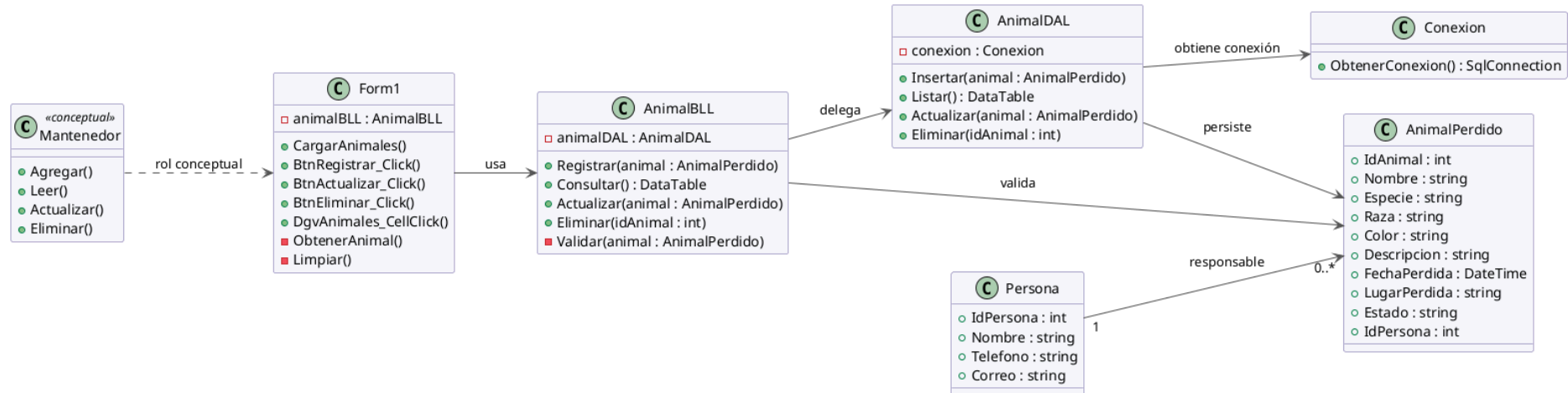


Figura 2. Diagrama de clases del sistema y abstracción conceptual del mantenedor.

7 7. Conclusión y referencias

El proyecto tiene una base funcional para registrar animales perdidos. Cuenta con una pantalla Windows Forms, capas separadas, validaciones y procedimientos almacenados. Este documento mantiene lo que realmente existe y no agrega funciones nuevas. La clase Mantenedor aparece de forma conceptual con `Agregar()`, `Leer()`, `Actualizar()` y `Eliminar()`, como pide el trabajo.

7.1 Referencias

- [1] [Repositorio GitHub del proyecto](<https://github.com/torrestorres329/Registro-Animales-Perdidos>), "Registro-Animales-Perdidos".
- [2] [ISO](<https://www.iso.org/standard/27001>), "ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements".